SOKOINE UNIVERSITY OF AGRICULTURE

School of Engineering and Technology
Department of Agricultural Engineering

AGE 219: Basics of Computer Programming

Lecture Compendium 6: Data Structures — Engineering Large-Scale Agricultural Systems

---

Prepared by:

Dr. Kadeghe Fue, PhD, P.Eng (T)

Target Audience: Engineering Students (Agricultural, Irrigation, Post-Harvest, and related fields)
Primary Tools: Python 3.x and Google Colaboratory
Location: Morogoro, Tanzania

Data is the lifeblood of modern agriculture. In this lecture, we move from processing individual numbers to engineering entire data systems. We will learn how to organize thousands of sensor readings, machinery parameters, and spatial field maps into coherent, efficient, and searchable structures using Python's Lists, Tuples, and Dictionaries.

Contents

1 Introduction: The Transition to Vector Engineering **2**
1.1 Why Data Structures Matter for SET Students 2

2 Python Lists: The Dynamic Conveyor Belt **2**
2.1 Internal Mechanics: How Lists Work in RAM 2
2.2 Indexing and Slicing: Precision Access 2
2.3 The Power of List Methods 3

3 List Comprehensions: The Engineer's Multi-tool **3**
3.1 The Functional Transformation Logic 3
3.2 Multispectral Data Processing Example 3

4 Tuples: The Immutable Reference **4**
4.1 Use Cases: GPS and Calibration 4

5 Dictionaries: Mapping Farm Relationships **4**
5.1 The "Hash Table" Concept 4
5.2 Metadata Management 4

6 Advanced Structures: Nesting and Matrices **5**
6.1 Matrices: Lists of Lists 5
6.2 Hierarchical Data Structures 5

7 Case Study 1: Fleet Telematics and Maintenance Hub **5**

8 Case Study 2: Spatial Yield Data Filtering **6**

9 Case Study 3: IoT Warehouse Inventory with Sets **6**

10 Case Study 4: ISOBUS Data Packet Parsing (Simplified) **6**

11 Best Practices for Agri-Engineers **7**

12 Theory of Algorithmic Complexity in Agricultural Data **7**
12.1 Linear Search vs. Hash Lookup 7

13 Summary **7**

14 Glossary of Terms **8**

15 Extended Self-Assessment Exercises **9**
15.1 Exercise 1: The Multi-Year Harvest Analyzer 9
15.2 Exercise 2: The Greenhouse Metadata Hub 9
15.3 Exercise 3: Coordinate Distance Calculation 9
15.4 Exercise 4: Complex Data Cleaning 9

1 Introduction: The Transition to Vector Engineering

In the previous modules of AGE 219, we focused on "Scalar" programming—handling one variable at a time. For a simple calculation, like the area of one plot, this is sufficient. However, a professional Agricultural Engineer at a large estate (like the Kilombero Sugar Company) or a researcher monitoring climate change impacts must handle "Vectors" or "Collections" of data.

Imagine a soil moisture network consisting of 500 sensors across a 1,000-hectare farm. If each sensor reports its value once every hour, the system generates 12,000 data points per day.

- **The Old Way:** Creating 500 variables (`sensor_1`, `sensor_2`, ...). This is impossible to manage.
- **The Engineering Way:** Using a **Data Structure** to store all 500 readings in a single container that can be sorted, filtered, and analyzed using a single loop.

1.1 Why Data Structures Matter for SET Students

In the School of Engineering and Technology (SET), our objective is **Efficiency**. A poorly chosen data structure can make a program slow, memory-intensive, or prone to errors. Choosing the right structure (e.g., a Dictionary for quick lookups vs. a List for ordered sequences) is as critical as choosing the right material for a bridge or the right pump for an irrigation system.

2 Python Lists: The Dynamic Conveyor Belt

A **List** is an ordered, mutable, and heterogeneous collection of items. In physical terms, think of a list as a modular conveyor belt in a post-harvest facility. You can place different items on it, change the order, add more modules to make it longer, or remove items as they reach the end of the line.

2.1 Internal Mechanics: How Lists Work in RAM

When you create a list, Python allocates a block of memory. Interestingly, Python usually "over-allocates" space. If you create a list of 4 items, Python might actually reserve space for 8 in the RAM. This is an engineering optimization: it allows you to `.append()` new items quickly without having to move the entire list to a new memory location every time.

As engineers, we must understand that lists are **dynamic arrays**. When a list exceeds its current allocated capacity, Python automatically allocates a larger block (usually double the size), copies the existing elements over, and deletes the old block. This ensures that while appends are generally very fast ($O(1)$ amortized), the memory usage can sometimes be higher than the actual number of elements.

2.2 Indexing and Slicing: Precision Access

Accessing data in a list is a "Constant Time" operation, meaning it is incredibly fast regardless of how long the list is.

```
# Daily Rainfall (mm) for the first two weeks of April
april_rain = [12.5, 0.0, 4.2, 0.0, 15.1, 8.0, 0.0, 2.0, 0.0, 0.0, 5.5, 10.2, 0.0, 1.5]

# Zero-based indexing
monday_rain = april_rain[0]

# Negative indexing (Very useful for 'Latest Data')
```

```
latest_rain = april_rain[-1]

# Slicing: [Start : Stop : Step]
# Extracting the first week (Index 0 to 6)
week_1 = april_rain[0:7]

# Extracting every other day (Stride)
every_other_day = april_rain[::2]
```

2.3 The Power of List Methods

As engineers, we often need to manipulate sequences. Python provides built-in tools for this:

- `.append(x)`: Add an item to the end (e.g., logging a new sensor reading).
- `.extend([x, y])`: Combine two lists (e.g., merging morning and afternoon data).
- `.sort()`: Organize data (e.g., finding the lowest and highest yields).
- `.pop()`: Remove and return an item (e.g., processing a queue of machinery tasks).

3 List Comprehensions: The Engineer's Multi-tool

List comprehensions provide a concise syntax for creating a new list based on the values of an existing list. This is much more than a "shorthand"—it is a mathematically elegant way to transform datasets.

3.1 The Functional Transformation Logic

In engineering, we often need to apply a transfer function to a set of raw signals. *Example: Converting 100 voltage readings from a pressure transducer into actual PSI values.*

```
# Raw Voltages from 0V to 5V
raw_voltages = [1.2, 2.5, 4.1, 0.5, 3.8]

# Transfer Function: PSI = (Voltage * 20.0) - 5.0
# We can do this transformation in one line:
psi_readings = [(v * 20.0) - 5.0 for v in raw_voltages]

# We can also add logic: Only keep readings above 10 PSI
high_pressure_only = [p for p in psi_readings if p > 10.0]

print(f"Calculated PSI: {psi_readings}")
```

3.2 Multispectral Data Processing Example

In precision agriculture, we often deal with multispectral data. Imagine we have a list of Red band values and a list of Near-Infrared (NIR) band values for different plots. We can use `zip()` and list comprehensions to calculate the NDVI (Normalized Difference Vegetation Index) for all plots at once.

```
red_values = [0.1, 0.2, 0.15, 0.3]
nir_values = [0.6, 0.7, 0.65, 0.4]

# Formula: (NIR - Red) / (NIR + Red)
ndvi_list = [(n - r) / (n + r) for r, n in zip(red_values, nir_values)]

print(f"Calculated NDVI values: {ndvi_list}")
```

4 Tuples: The Immutable Reference

A **Tuple** is an ordered, **immutable** collection. Once defined, it cannot be changed. In the engineering world, we use tuples to store ”Truths” or ”Constants.”

4.1 Use Cases: GPS and Calibration

- **GPS Coordinates:** The latitude and longitude of a permanent weather station should not be modified by a program error.
  - Example: `sua_station = (-6.8485, 37.6621)`
- **Color Codes:** RGB values for identifying different crop diseases in image processing.
- **Return Values:** Functions that need to return multiple pieces of data (e.g., returning both the Mean and Standard Deviation of a soil sample).

```
# Fixed GPS coordinates for SUA Meteorological Station
station_loc = (-6.8485, 37.6621)

# Attempting to change a tuple will cause a TypeError
# station_loc[0] = -7.0 # This would protect your data from accidental modification
```

5 Dictionaries: Mapping Farm Relationships

While lists are ordered by numbers (0, 1, 2...), **Dictionaries** are ordered by **Keys**. A dictionary is a collection of `Key:Value` pairs. This is arguably the most powerful data structure for Digital Agriculture.

5.1 The ”Hash Table” Concept

Internally, Python uses a ”Hash Table” for dictionaries. This means that looking up a value for a specific key (like `sensor_data['BLOCK_A_TEMP']`) is almost instantaneous, even if the dictionary contains 1,000,000 entries. This is achieved by converting the key into a hash (a unique integer) that maps directly to a memory address.

5.2 Metadata Management

We use dictionaries to store information that is descriptive rather than just sequential.

```
# Describing a specific tractor in the SUA fleet
tractor_01 = {
    "brand": "Massey Ferguson",
    "model": "MF 4700",
    "hp": 85,
    "last_service_hours": 450.5,
    "is_active": True
}

# Accessing values via keys
current_hp = tractor_01["hp"]

# Adding new data
tractor_01["operator_name"] = "John Juma"

# Safe access with .get() - prevents the program from crashing if key is missing
fuel_level = tractor_01.get("fuel_level", "UNKNOWN")
```

6 Advanced Structures: Nesting and Matrices

Real-world engineering data is multi-dimensional. We handle this by nesting one data structure inside another.

6.1 Matrices: Lists of Lists

A field can be modeled as a grid. Each sub-list represents a row of soil moisture sensors. This is essentially a two-dimensional array.

```
# 3x3 Grid of Soil Moisture Sensors (%)
field_grid = [
    [22, 25, 21],  # Row 0
    [19, 20, 18],  # Row 1
    [30, 31, 28]   # Row 2
]

# Accessing the center sensor (Row 1, Column 1)
center_moisture = field_grid[1][1]

# Calculating row averages
row_averages = [sum(row)/len(row) for row in field_grid]
```

6.2 Hierarchical Data Structures

In large-scale management, we might use a dictionary where each key represents a farm block, and each value is another dictionary containing sensor arrays.

```
farm_management = {
    "North_Block": {
        "crop": "Maize",
        "sensors": [25, 26, 24, 28],
        "irrigation_active": True
    },
    "South_Block": {
        "crop": "Rice",
        "sensors": [32, 35, 31, 33],
        "irrigation_active": False
    }
}
```

7 Case Study 1: Fleet Telematics and Maintenance Hub

Engineering Problem: A farm manager needs a system to track the health of a tractor fleet. Each tractor has a unique ID, a brand, and an accumulated hour meter. The system must flag tractors that are overdue for a 500-hour service.

```
# Fleet data structure: Nested Dictionary
fleet = {
    "TRACTOR_A": {"brand": "John Deere", "hours": 480},
    "TRACTOR_B": {"brand": "Massey Ferguson", "hours": 515},
    "TRACTOR_C": {"brand": "New Holland", "hours": 200}
}

print("--- Maintenance Alert System ---")
for tid, data in fleet.items():
    current_hours = data["hours"]
    if current_hours >= 500:
```

```
        overdue = current_hours - 500
        print(f"ALERT: {tid} ({data['brand']}) is {overdue} hours OVERDUE for service!")
    else:
        remaining = 500 - current_hours
        print(f"STATUS: {tid} has {remaining} hours until next service.")
```

8 Case Study 2: Spatial Yield Data Filtering

Engineering Problem: A combine harvester generates yield data in a 2D list. We need to find the "Low Yield Zones" (where yield is below 2.0 tons/ha) so we can investigate soil nutrient deficiencies.

```
# Yield data (Tons/Ha) across 4 rows and 5 sub-plots
yield_map = [
    [4.5, 4.2, 3.8, 1.5, 4.0],
    [3.9, 1.2, 4.1, 4.4, 4.3],
    [4.6, 4.5, 1.8, 3.7, 4.1],
    [4.0, 3.8, 4.2, 4.1, 4.5]
]

print("Scanning for Low Yield Coordinates...")
for r_idx, row in enumerate(yield_map):
    for c_idx, val in enumerate(row):
        if val < 2.0:
            print(f"LOW YIELD: {val} t/ha found at Row {r_idx}, Plot {c_idx}")
```

9 Case Study 3: IoT Warehouse Inventory with Sets

Engineering Problem: A warehouse receives different varieties of seed and fertilizer. We want to identify the **unique** set of products currently in stock across three different storage silos.

```
silo_1 = {"Maize-H624", "Urea", "DAP"}
silo_2 = {"Rice-SARO5", "Urea", "Potash"}
silo_3 = {"Maize-H624", "Beans-Yellow", "DAP"}

# Union of all sets to find total unique product types
all_products = silo_1 | silo_2 | silo_3

print(f"Total Unique Product Varieties in Stock: {len(all_products)}")
print(f"Inventory List: {all_products}")

# Intersection to find products common to ALL silos
common_products = silo_1 & silo_2 & silo_3
print(f"Products available in every silo: {common_products}")
```

10 Case Study 4: ISOBUS Data Packet Parsing (Simplified)

In modern tractors, machinery communicates via the ISOBUS standard (ISO 11783). Data packets often contain hex codes mapped to specific machine parameters. While we won't parse binary here, we can simulate the lookup table using a dictionary.

```
# Mapping PGNs (Parameter Group Numbers) to Descriptions
isobus_pgns = {
    61444: "Electronic Engine Controller 1",
    65262: "Engine Temperature",
    65263: "Engine Fluid Level/Pressure",
```

```
    65266: "Fuel Economy (Liquid)"
}

# Simulating an incoming data stream
incoming_packet_pgn = 65262
parameter = isobus_pgns.get(incoming_packet_pgn, "Unknown PGN")
print(f"Incoming Data Group: {parameter}")
```

11 Best Practices for Agri-Engineers

1. **Type Consistency:** Although Python allows a list like `[1, "Maize", 3.14]`, it is bad practice. Keep lists homogeneous (all items the same type) for predictable engineering simulations.
2. **Immutability by Design:** Use **Tuples** for data that *should not* change. If your code modifies a GPS coordinate by mistake, it is much better for the program to crash (Tuple) than to proceed with wrong coordinates (List).
3. **Dictionary Key Choice:** Always use strings or integers as dictionary keys. Avoid using floats, as their precision errors (e.g., `0.1 + 0.2`) can make them impossible to find.
4. **The Efficiency Principle:** If you need to search for a value millions of times (e.g., looking up a crop coefficient based on a crop ID), use a **Dictionary**. Searching a list of 1,000,000 items takes a long time ($O(n)$); searching a dictionary of 1,000,000 items is nearly instant ($O(1)$).

12 Theory of Algorithmic Complexity in Agricultural Data

As engineers, we must care about the performance of our code when data scales.

12.1 Linear Search vs. Hash Lookup

Consider searching for a specific Tractor Serial Number in a list of 10,000 tractors. In the worst case, Python has to check every single item. This is called $O(n)$ complexity. If you use a dictionary with the Serial Number as the key, Python uses a hash function to jump directly to the memory address where the tractor data is stored. This is $O(1)$ complexity.

For a system that processes real-time sensor data from thousands of nodes every second, choosing $O(1)$ over $O(n)$ is the difference between a functional system and a crashed one.

13 Summary

Data structures are the architectural foundation of your programs. By choosing between Lists (for order), Tuples (for safety), and Dictionaries (for relationships), you can model complex agricultural ecosystems with minimal code. In the next lecture, we will learn how to make these structures **Permanent** by saving them to and loading them from **External Files (CSV and Text)**.

14 Glossary of Terms

- **Algorithm Complexity ($O(n)$):** A notation describing how the execution time of a task increases as the amount of data increases.
- **Dictionary Comprehension:** A shorthand for creating dictionaries, similar to list comprehensions.
- **Dynamic Array:** The underlying structure of a Python list that allows it to grow or shrink in size during execution.
- **Hash Table:** The underlying technology behind dictionaries that allows for near-instant data retrieval using keys.
- **Heterogeneous:** A collection containing different types of data (e.g., integers and strings).
- **Immutable:** An object that cannot be modified after creation (Tuples, Strings).
- **Mapping:** A relationship where a unique key is associated with a specific value.
- **Matrix:** A two-dimensional array of numbers, represented in Python as a list of lists.
- **Mutable:** An object that can be modified after creation (Lists, Dictionaries).
- **Sequence:** An ordered collection of items where position matters (Lists, Tuples).

15 Extended Self-Assessment Exercises

15.1 Exercise 1: The Multi-Year Harvest Analyzer

A farmer recorded maize yields (tons) for the last 10 years: `yields = [4.5, 4.8, 2.1, 5.0, 4.9, 1.5, 4.2, 4.7, 4.3, 4.1]` **Task:** 1. Slicing: Create a list of the ”Recent 5 Years” of data. 2. Filtering: Use a list comprehension to create a list of ”Drought Years” (yield < 3.0 tons). 3. Transformation: Create a list of ”Percentage of Peak” where each yield is expressed as a percentage of the highest yield in the 10-year period.

15.2 Exercise 2: The Greenhouse Metadata Hub

Create a dictionary representing a smart greenhouse. The dictionary should contain:

- `location`: A tuple of (Lat, Lon).
- `sensors`: A list of strings (IDs).
- `status`: A boolean (True if active).

Task: Write a script that checks if the greenhouse is active, and if so, prints the total number of sensors it manages.

15.3 Exercise 3: Coordinate Distance Calculation

Given two tuples representing farm boundary points: `P1 = (10, 20)` and `P2 = (40, 60)`. **Task:** Write a script that calculates the Euclidean distance between these points using the formula:

$$d = \sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2}$$

(Hint: You will need to import the `math` module).

15.4 Exercise 4: Complex Data Cleaning

An IoT node sometimes reports an error code instead of a temperature: `raw_logs = [22.1, 23.5, "ERR_404", 22.8, 21.9, "ERR_500", 24.0]` **Task:** Use a list comprehension to create a ”Clean Log” that only includes the floating-point numbers. Then, calculate the average temperature of the clean log.

End of Lecture 6 Compendium